CONSIGNA TRABAJO INTEGRADOR:
En este proyecto el alumno deberá tener en cuenta los siguientes puntos
para realizar el desarrollo solicitado:1. Requisitos Funcionales

- Registro e Inicio de Sesión:
- Implementar autenticación con opciones de registro e inicio de sesión
- Autenticación mediante 3 redes sociales (Google, Facebook, ETC).

Roles de Usuario:

- Múltiples tipos de usuarios con distintos permisos (Minimo 3)
- Comportamiento y accesos específicos dependiendo del rol/perfil.

Interfaz de Usuario (UI):

- Una interfaz amigable y fácil de navegar que se adapte a dispositivos móviles.
- Navegación entre diferentes pantallas y componentes de React Native bien estructurados.

Consumo de API:

- Llamadas al backend usando una API RESTful que devuelva datos y permita crear, leer, actualizar y eliminar registros (CRUD/ABML).
- La aplicación debe manejar correctamente los errores y mostrar mensajes adecuados a los usuarios.

2. Requisitos Técnicos de Frontend (React Native)

- Navegación:
- Usar un sistema de navegación adecuado, que permita mover entre pantallas de forma fluida.

Estado y Contexto:

- Usar Redux o el Context API para gestionar el estado global, especialmente para gestionar el login y los datos del usuario.

Autenticación Persistente:

- Implementar persistencia de la sesión del usuario, para que el usuario siga conectado tras cerrar y volver a abrir la app.

Componentes Responsivos:

- Diseñar componentes que se adapten a diferentes tamaños de pantalla y orientación del dispositivo.

Manejo de Errores:

- Implementar manejo de errores en formularios y en la conexión con el backend.

3. Requisitos Técnicos de Backend

El backend puede ser Node.js con Express o Django en Python, PHP o el que mejor maneje. Debería incluir:

- API REST:
- Implementación
  de una API RESTful que permita la comunicación con la app. La API
  debería incluir endpoints para login, registro, obtención y
  actualización de datos.
- Autenticación basada en tokens (JWT), con tokens de acceso y renovación.

Base de Datos:

- Una base de datos MySQL, bien estructurada con colecciones o tablas para los usuarios y cualquier otra entidad necesaria.

Gestión de Roles:

- Implementar
  lógica para determinar permisos de acuerdo al rol del usuario,
  protegiendo endpoints que no deberían ser accesibles para todos.

Documentación del API:

- Documentación de los endpoints de la API, que incluya los métodos HTTP usados, parámetros, respuestas y códigos de error.
- Usar un archivo README para la documentación de la API.

4. Buenas Prácticas de Código

- Limpieza y Estructura:
- Código bien organizado y fácil de leer, con documentación donde sea necesario.

Organización del Proyecto:

- Estructura de carpetas lógica tanto en el frontend como en el backend.

Control de Versiones:

- Uso
  de Git y GitHub (o similar) para el control de versiones, con commits
  claros y un README que explique cómo instalar y ejecutar el proyecto.

5. Entregables y Evaluación

- Código Fuente Completo:
- El código fuente completo de la aplicación frontend y el backend en un repositorio de Git.

Manual de Usuario:

- Un documento con instrucciones sobre cómo usar la aplicación (iniciar sesión, navegar entre pantallas, etc.).

Guía de Instalación y Configuración:

- Instrucciones
  claras para instalar y configurar el entorno de desarrollo, tanto para el frontend como para el backend (instalación de dependencias, variables de entorno, ejecución de la base de datos, etc.).

Documentación de la API:

- Documentación de los endpoints y cualquier otro detalle técnico necesario para entender cómo interactuar con el backend.

Video de Demostración (opcional):

- Un
  breve video mostrando la aplicación en uso, en el que se vea la funcionalidad de inicio de sesión, navegación, distintos roles de usuario y consumo de API.

APK

- Deberá contar con un Archivo APK(android) para poder descargar, instalar y probar.

rEQUERIMIENTOS GENERALES::

A continuación se listarán los requerimientos que se deberán cumplir a lo largo de la completitud de proyectos a entregar:

- Funcionalidad
  - Completitud
  - Debe cumplir con lo solicitado en el proyecto
  - Sin errores (Aplica a cada agregado a lo ya solicitado)
  - Adaptabilidad (Responsive)
- Escalabilidad
  - Modularidad
  - Atomización
  - Versionado
  - Documentación
  - Código
    - Prolijidad
    - Identación
    - Conocimiento del mismo
    - Buenas formas de desarrollo
- Normas
  - Estructuración
  - Validación
  - Usabilidad
- Importancia
  - Usabilidad
    - Claro-Oscuro

        - Aprovechamiento de pantalla (sin espacios huecos a los costados en desktop)
        - Orden de los objetos en pantalla
    - Lógica
    - UX/UI (Interfaces bonitas, agradables e intuitivas)
    - Tiempos y formas de la entrega
- BBDD
    - Seguridad
    - Mínimo 3 forma de normalización
    - Buenas formas
    - Relacional SQL

- Prohibidos los alerts, en su lugar usar modales estéticos
- los comentarios de codigo deben estar EN TODO EL PROYECTO, con comentarios en 1era persona donde corresponda, y todo en minuscula por ejemplo //acá la funcion hace…. o //funcion “tanto”, para hacer… Ademas deben estar en todo el codigo, al principio como introductorio, y antes de cada funcion y ciclo TODO par aleer y entender rapido
- NO QUIERO CODIGO DE MAS D E400 LINEAS COMO MUCHO. QUIERO MODULARIZACION Y CARPETAS CLARAS
- CARPETAS: SEPARA FRONTEND DE BACKEND CON SUS RESPECTIVOS CONTENIDOS.
- SIEMPRE implementar el botón de volver arriba del todo (tipo sticky top) cuando el contenido es muy largo.
- Para El tp final, se usará expo go, react native, bd maria db, heidi sql, aiven para host publico (si aplica)
-